Docker basics

Containers, Dockerfile, best practices, multi-arch

Course: DevOps Pro — Kubernetes & CI/CD on GitLab

Author: Aliaksandr Tsimokhau

Agenda

Part 1 — Containerization basics

History, mechanics (namespaces, cgroups, runtimes), VM vs container

Part 2 — Images & layers

UnionFS pancake stack, Copy-on-Write, Btrfs

Part 3 — Dockerfile fundamentals

Anatomy of a Dockerfile, base example

Part 4 — Dockerfile best practices

Versioning, USER, minimal base, multi-stage, layer hygiene

Part 5 — Key instructions deep-dive

ARG vs ENV, ADD vs COPY, RUN/CMD/ENTRYPOINT, exec vs shell

Part 6 — Docker/Podman CLI essentials

Build, pull/push, login, tag, run, exec, logs, inspect, cleanup

Part 7 — Multi-arch builds

Buildx, QEMU, build & push for amd64/arm64/...

Part 8 — Hands-on

Improve a real Dockerfile

Part 1 — Containerization basics

Why containers exist and how they differ from VMs

A short history of isolation

- **1979** — `chroot()` system call in UNIX: change root `/` for a group of processes
- **2000** — FreeBSD jails: partial isolation of network interfaces
- **2002** — Linux `mount` namespace lands in the kernel
- **2006-2007** — Google's Process Containers → renamed to `cgroups`
- **2008** — `cgroups` upstreamed into Linux; LXC released the same year
- **2013** — User namespace completes the isolation primitives; Docker is released
- **2015** — Open Container Initiative (OCI) standardizes the ecosystem

`chroot` (1979) — the first "jail"

- "Change Root" — replaces the apparent `/` for a process and its children
- A process inside the jail **cannot see files outside** the designated tree

Limitation: `chroot` only isolates the **filesystem**

- It does **not** isolate processes, networking, or users
- A root process inside the jail could still kill processes outside

→ First step toward isolation, but true containerization needs more.

Namespaces — the illusion of privacy

Linux kernel feature: **partition kernel resources** so different process groups see different sets.

Namespace	What it isolates
PID	Process tree — container thinks it's PID 1 (host: PID 14500)
NET	Own virtual IP, ports, routing table
MNT	Own filesystem mount tree
UTS	Own hostname
USER	Own UID/GID mappings (root inside ≠ root outside)
IPC	Own message queues, shared memory

→ **Isolation** — the kernel lies to the process about its environment.

Cgroups — the resource police

Google's "Process Containers" (2006) → upstreamed into Linux as `cgroups`.

Function: limit, account for, and isolate resource usage of a process group:

- CPU shares & quota
- Memory limit
- Disk I/O (block I/O)
- Network bandwidth

Without cgroups: one container running a hot loop pins 100% of CPU and freezes everything else.

OOM Killer: when a container exceeds its memory limit, the kernel kills *that specific process* to save the system.

→ **Limitation** — ensures fair sharing of hardware.

Container runtime hierarchy — it's not just "Docker"

Docker is a **stack of tools**, not a single binary.

High-level (managers)

- Docker Engine
- Podman
- CRI-O

Role: human-facing — pull images, manage volumes, networks, expose the API.

Low-level (workers)

- `runc` (standard, OCI reference)
- `crun` (fast, C-based)
- `gVisor` (sandboxed, user-space kernel)

Role: talk to the kernel — set up namespaces, apply cgroups, exec the process.

> `runc` creates the container and then **exits**, leaving the process running under `containerd-shim`.

So what is a container, really?

The formula

```
Namespaces  (isolation)
    +
Cgroups     (resource limits)
    +
UnionFS     (layered images)
    =
Container
```

The hotel analogy

- **Your own door** — Namespaces
- **Limit on electricity & room service** — Cgroups
- **Plumbing, foundation, power** — Host kernel

A **VM** is the same idea, but a **fully built house on its own plot** — heavier, slower, fully separate.

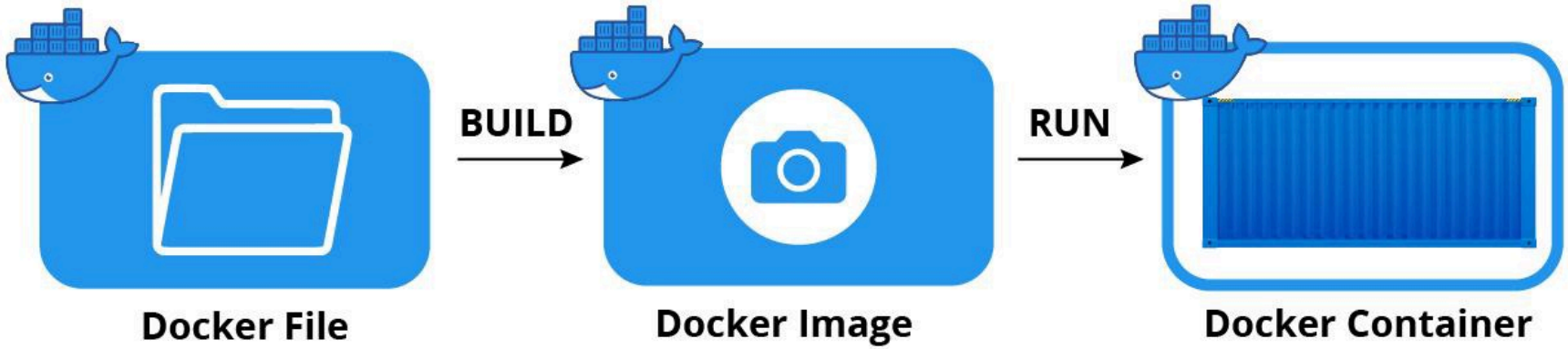
Seven principles of containerization

1. **One concern per container** — single responsibility → scalability & reuse
2. **Image is immutable** — all changes go into the build, not into the running container
3. **Disposability** — kill and replace any container at any moment
4. **Observability** — readiness/liveness markers, logs to standard streams
5. **Manageability** — container reacts cleanly to external signals (graceful shutdown)
6. **Self-containment** — image carries all deps; orchestrators handle inter-container links
7. **Resource limits** — set CPU/memory bounds, prioritize, avoid noisy-neighbor

Virtual machines vs containers



Dockerfile, image, container — the photo analogy



- **Dockerfile** — recipe for the photo (camera settings, location, subject)
- **Image** — the actual photo created by following the recipe (read-only, shareable)
- **Container** — a printed copy you hold in your hand (running instance, isolated)

Part 2 — Images & layers

The pancake stack, Copy-on-Write, and what's actually on disk

Image layers — the "pancake stack" (UnionFS)



Which directives create a layer?

⊕ Adds a filesystem layer

- `FROM` — base image's layers
- `RUN` — filesystem changes from the command
- `COPY` — files from the build context
- `ADD` — like `COPY` (+ URL / tar extraction)

These add **bytes** to the image.

· Metadata-only (no FS layer)

- `CMD`, `ENTRYPOINT` — what runs at start
- `ENV`, `ARG` — env & build-time vars
- `WORKDIR`, `USER` — directory & user
- `EXPOSE`, `LABEL` — docs / metadata
- `VOLUME`, `STOPSIGNAL`, `HEALTHCHECK`, `SHELL`, `ONBUILD`

Zero bytes on disk — pure config in the image manifest.

```
FROM python:3.12-slim                # ⊕ layer (base)
WORKDIR /app                         # · metadata
ENV PYTHONDONTWRITEBYTECODE=1        # · metadata
COPY requirements.txt .              # ⊕ layer
RUN pip install -r requirements.txt  # ⊕ layer
COPY src/ .                          # ⊕ layer
USER 1000                            # · metadata
CMD ["python", "main.py"]            # · metadata
```

> ⚠ Cache invalidation still happens on **every** instruction — change a metadata line and everything below it rebuilds.

Copy-on-Write — the secret to speed

How it works:

- **Read** — the app reads a file → served directly from the underlying image layer (fast, no copy)
- **Write** — the app modifies a file → the storage driver performs a **copy-up**: the file is copied from a read-only layer to the writable top layer, then modified there

Benefits:

- Container starts in **milliseconds** — no full filesystem copy required
- Disk usage minimized — a 1 GB image may take only **1 MB** of extra disk per container if few files change

> 💡 Don't store state in the writable layer — it dies with the container. Use **volumes** for persistence.

Btrfs — native CoW filesystem (advanced)

Modern Linux FS with **built-in volume management** — like ext4/XFS, but the filesystem *itself* does Copy-on-Write.

`overlay2` is Docker's default storage driver; **Btrfs** handles layering **natively at the filesystem level** — no extra driver needed.

Key features:

- **Subvolumes** — act like separate disks; Docker uses one per container
- **Snapshots** — instant "photo" of FS state → fast `docker build` & `docker commit`
- **Data integrity** — checksums on data + metadata catch silent corruption

Part 3 — Dockerfile fundamentals

Anatomy of a Dockerfile and the core instructions

Dockerfile example

```
FROM python:3.12
WORKDIR /usr/local/app

# Install the application dependencies
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

# Copy in the source code
COPY src ./src
EXPOSE 5000

# Run as a non-root user
RUN useradd app
USER app

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Part 4 — Dockerfile best practices

Versioning, security, layer hygiene, cache

Best practices — versioning

1. Use the current release base upstream image — security patches, latest features.

2. Pin a specific image tag

```
FROM python:3.12
```

3. Pin a specific dependency version

```
# Avoid
RUN pip install some-package
# Prefer
RUN pip install some-package==1.2.3
```

Best practices — security & size

4. Run as a non-root USER

```
RUN useradd -m appuser
USER appuser
```

5. Choose a minimal base image — avoid full OS

```
FROM python:3.12-slim
```

Best practices — multi-stage builds

6. Separate build and runtime environments

```
FROM python:3.12 AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /app /app
COPY src ./src
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Best practices — ignore & scan

7. Use a `.dockerignore` file

```
# .dockerignore
.git
__pycache__
*.pyc
```

8. Scan for vulnerabilities

```
docker scan myimage:latest
```

Best practices — order for cache

9. Order Dockerfile instructions to leverage cache

```
FROM python:3.12
WORKDIR /app

# Dependencies first — change rarely
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Source last — changes most often
COPY src ./src

EXPOSE 5000
RUN useradd -m appuser
USER appuser

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Best practices — layer hygiene

10. Concatenate `RUN` commands.
11. Clean up package manager caches in the same layer.

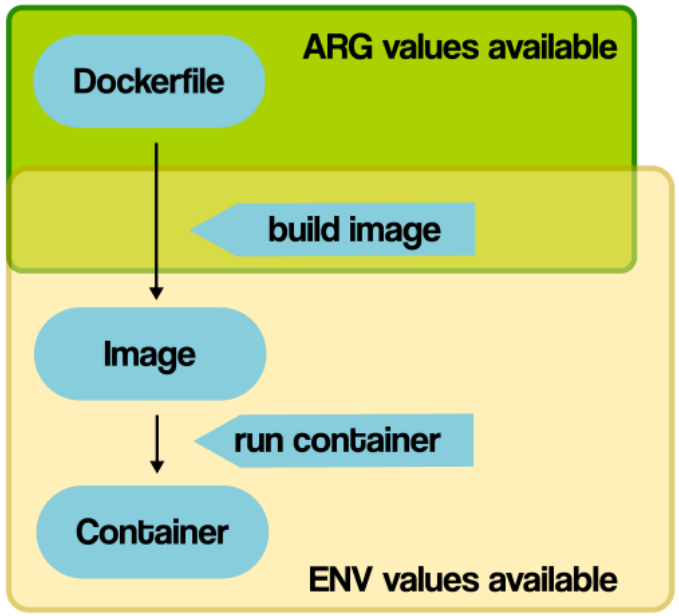
```
# Avoid — leaves apt cache in the layer
RUN apt-get update && apt-get install -y \
    package1 \
    package2

# Prefer — single layer, no leftovers
RUN apt-get update && apt-get install -y \
      package1 \
      package2 \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*
```

Part 5 — Key instructions deep-dive

ARG vs ENV, ADD vs COPY, RUN/CMD/ENTRYPOINT, exec vs shell

ARG vs ENV



Dockerfile content:

```
ARG required_var       # expects a value
ARG var_name=def_value # set default ARG value
ENV foo=other_value    # set default ENV value
ENV bar=${var_name}    # set default ENV value from ARG
```

Override Dockerfile ARG values on build:

```
$ docker build --build-arg var_name=value
```

Override Docker image ENV values on run:

```
$ docker run -e "foo=other_value" [...]
$ docker run -e foo [...] # pass from host
$ docker run --env-file=env_file_name [...] # pass from file
```

https://vsupalov.com/docker-env-vars

ADD vs COPY


Start
What's the source?
files / tar / URL
Files in context
Local tar
Remote URL
COPY

RUN, CMD, ENTRYPOINT



Exec form vs shell form

```
# Exec form — preferred
CMD ["python", "app.py"]
ENTRYPOINT ["python", "app.py"]
```

```
# Shell form — wraps in /bin/sh -c
CMD python app.py
ENTRYPOINT python app.py
```

Exec vs shell — decision tree



Part 6 — Docker/Podman CLI essentials

Build, run, inspect, debug, publish, and clean up containers

CLI mental model

The CLI moves between **three states**: source code, image, running container.

State	Create with	See with	Remove with
Image	`build`, `pull`	`images`, `history`, `inspect`	`rmi`
Container	`run`, `create`	`ps`, `logs`, `inspect`, `stats`	`rm`
Registry copy	`push`	registry UI / `pull`	registry cleanup policy

> Podman uses the same OCI image format. Most commands are `docker ...` → `podman ...`.

Build an image

`build` reads a Dockerfile and build context, then stores a local image tag.

```
# Docker
docker build -f Dockerfile -t demo-api:1.0.0 .
docker images demo-api
docker history demo-api:1.0.0

# Podman
podman build -f Dockerfile -t demo-api:1.0.0 .
podman images demo-api
podman history demo-api:1.0.0
```

`-t name:tag` is the local name you will use in `run`, `tag`, or `push`.

Run vs create

`run` creates **and starts** a container. `create` only prepares it.

```
# One-shot: remove container after command exits
docker run --rm alpine:3.20 echo "hello"
podman run --rm alpine:3.20 echo "hello"

# Service-style: detached, named, port published
docker run -d --name web -p 8080:8080 demo-api:1.0.0
podman run -d --name web -p 8080:8080 demo-api:1.0.0

# Create now, start later
docker create --name job demo-api:1.0.0
docker start job
```

Stop and delete containers

Containers are disposable. Stop them cleanly, then remove them.

```
# List running and stopped containers
docker ps
docker ps -a

# Graceful stop, then delete
docker stop web
docker rm web

# Delete a stopped container automatically after a command
docker run --rm demo-api:1.0.0 pytest
```

Podman equivalent: replace `docker` with `podman`.

Pull, login, tag, push

Registry workflow: authenticate, fetch base images, name your image, publish it.

```
# Authenticate to a registry
docker login registry.example.com
podman login registry.example.com

# Pull an upstream/base image
docker pull python:3.12-slim

# Add the registry-qualified tag
docker tag demo-api:1.0.0 \
  registry.example.com/platform/demo-api:1.0.0

# Publish the image
docker push registry.example.com/platform/demo-api:1.0.0
```

Describe what exists: inspect

Docker/Podman do not have `describe`. The equivalent is `inspect`.

```
# Container status, image, ports, mounts, env, networks
docker inspect web

# Focus the output with Go templates
docker inspect web \
  --format '{{.State.Status}} {{.NetworkSettings.IPAddress}}'

# Image config: default command, user, labels, env
docker inspect demo-api:1.0.0 \
  --format '{{.Config.User}} {{.Config.Cmd}}'
```

Use `history` for layers, `inspect` for metadata.

Logs and shell access

Containers should log to stdout/stderr, then `logs` can collect everything.

```
# Follow the last 100 log lines
docker logs -f --tail=100 web

# Run a command inside a running container
docker exec -it web sh

# Copy a file out for debugging
docker cp web:/app/report.txt ./report.txt
```

No shell in distroless images? Use app logs, `inspect`, or an external debug container.

Cleanup commands

Delete the right object: container first, image second, cache last.

```
# Remove stopped containers and unused images
docker container prune
docker image prune

# Remove one image tag
docker rmi demo-api:1.0.0

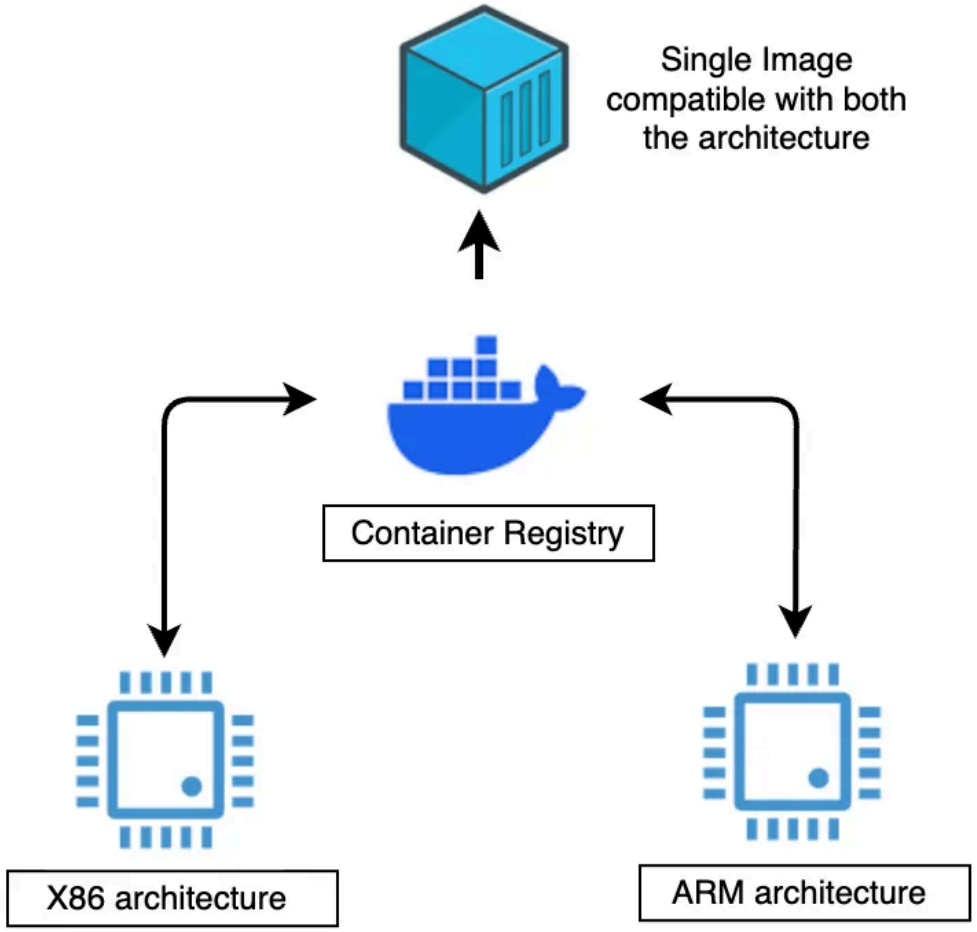
# See and reclaim build/cache disk usage
docker system df
docker builder prune
docker system prune
```

Podman equivalent: `podman system df`, `podman system prune`, `podman rmi`.

Part 7 — Multi-arch builds

One image, many CPU architectures

Multi-arch builds — the picture



Multi-arch — key concepts

- **Architectures** — `amd64` (x86_64), `arm64` (ARM 64-bit), `arm/v7` (ARM 32-bit), `ppc64le`, `s390x`, `riscv64`
- **Buildx** — Docker CLI plugin extending `docker build` with multi-arch capabilities (BuildKit under the hood)
- **QEMU** — open-source emulator that lets your machine execute binaries for foreign architectures

Multi-arch — build & push flow

1. **Install and enable Buildx** — extend Docker's build capabilities
2. **Enable QEMU** — emulate foreign architectures on your machine
3. **Create a multi-arch builder** — `docker buildx create --use`
4. **Write a Dockerfile** — same as usual, no special syntax needed
5. **Build and push** — `docker buildx build --platform linux/amd64,linux/arm64 -t … --push .`

```
docker buildx build \
  --platform linux/amd64,linux/arm64,linux/arm/v7 \
  -t registry.example.com/myapp:1.0.0 \
  --push .
```

Part 8 — Hands-on practice

Take this Dockerfile and make it production-ready

Starter Dockerfile — what's wrong?

```
FROM debian
# Copy application files
COPY . /app
# Install required system packages
RUN apt-get update
RUN apt-get -y install imagemagick curl software-properties-common gnupg vim ssh
RUN curl -sL https://deb.nodesource.com/setup_12.x | bash -
RUN apt-get -y install nodejs
# Install NPM dependencies
RUN npm install --prefix /app
EXPOSE 80
CMD ["npm", "start", "--prefix", "app"]
```

Checklist — apply what you've learned

- [ ] Pin a specific base image tag (no `latest`)
- [ ] Switch to a minimal base (`-slim`, `alpine`, or `distroless`)
- [ ] Merge `apt-get update && install` into one `RUN`, clean caches in the same layer
- [ ] Use `--no-install-recommends` and drop dev tools (`vim`, `ssh`)
- [ ] Reorder: dependencies before source code — cache stays warm
- [ ] Add `.dockerignore` for `node_modules`, `.git`, logs
- [ ] Introduce a multi-stage build — runtime stage contains only what's needed at run-time
- [ ] Create and switch to a non-root user
- [ ] Pick a non-privileged port (e.g., 8080)

Thank you!

Questions, feedback, or stuck on the practice?